



Full-stack software development

Front-end advanced: client-side Forms

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Forms

- React components that contain:
 - A form with input fields
 - Validation and error handling
 - Event handlers
- Every input field is bound to a state variable.
 - The form component can be re-rendered multiple times and you want to retain what the user already has entered.

Forms: rendering

```
<form onSubmit={handleSubmit}>
  <div className="row">
    <div className="col-md-1 offset-md-4">
      <label htmlFor="nameInput">Username:</label>
    </div>
    <div className="col-md-3">
      <input
        id="nameInput"
        type="text"
        value={name}
        onChange={(event) => setName(event.target.value)}
      />
      {nameError && <div className="text-danger">{nameError}</div>}
    </div>
  </div>

  <div className="row">
    <button
      className="btn btn-primary mt-3 col-md-1 offset-md-4"
      type="submit"
    >
      Login
    </button>
  </div>
</form>
```

Register an onSubmit event handler: when submitting the form, execute the handleSubmit function

Bind the state variable "name" to the input field.

Register onChange event handler: when the value of the input field changes, also update the state.

Conditional rendering: when nameError contains a value, render a div with that value.

Forms: declaring state

```
const [name, setName] = useState("");  
const [nameError, setNameError] = useState("");  
const [statusMessage, setStatusMessage] = useState<StatusMessage>(null);
```

Declare state for each field that can contain validation errors

For general status messages (error or success) declare a separate state field that is displayed above the form.
StatusMessage is a **custom** type that contains the fields "type" and "message".

Forms: validating

```
const validate = (): boolean => {  
  setNameError("");  
  setStatusMessage(null);  
  
  if (!name && name.trim() === "") {  
    setNameError("Name cannot be empty.");  
    return false;  
  }  
  
  return true;  
};
```

Reset previous state.

Because of state update, the component will be re-rendered and the div with the name error will be visible now.

Forms: submitting

```
const handleSubmit = async (event) => {  
  event.preventDefault();  
  
  if (!validate()) {  
    return;  
  }  
  
  const response = await UserService.loginUser({ username: name });  
  const data = await response.json();  
  
  if (response.status === 200) {  
    sessionStorage.setItem("username", name);  
    setStatusMessage({ type: "success", message: data.message });  
    setTimeout(() => {  
      router.push("/");  
    }, 2000);  
  } else if (response.status === 401) {  
    setStatusMessage({ type: "error", message: data.errorMessage });  
  }  
};
```

Prevent default browser behaviour when submitting forms: a full-page refresh.

Save the username in the session storage of the browser.

Display the “success” message and after 2 seconds, redirect to the home page. “router” is a variable that we initialized using the Next.js custom hook useRouter:

```
const router = useRouter();
```